

Lab1:DataLab 何梓昕 25303050286

截图：

```
roundEvenPow2: 11/24 operators
midpointTowardFirst: 24/32 operators
isBetweenEitherOrder: 30/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 35/52 operators
floatScaleThreeHalves: 44/60 operators
floatRoundEven: 46/65 operators
float_i2f: 33/40 operators
bitCount: 34/40 operators
bitReverse: 32/34 operators
All 19 functions passed operator checks.
```

```
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
hzixin@Bu:/mnt/c/Users/honor/Desktop/DataLab_hzixin$
```

实验思路（详细分类总结见后文）：

P1 signMask: 利用 $1 \ll 31$ 构造最高位为 1 的掩码 0x80000000。

P2 bitXor: 利用德摩根定律，将异或等价转化为 $\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$ 。

P3 negativePart: 通过 $x \gg 31$ 获取全 1 或全 0 的符号掩码。利用补码求相反数公式 $\sim x + 1$ ，结合掩码 & 实现“负数返回相反数，正数返回 0”。

P4 copyByteWithin: 先提取 src 字节，再将 dst 字节清零，最后通过 | 拼装。

P5 logicalShift: 通过构造掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 将算术右移补进来的高 n 位 1 擦除。

P6 swapNibblePairs: 利用 0x0F 逐级左移拼凑出 0x0F0F0F0F 掩码, 分别提取高低 4 位并交换移位, 最后用 | 拼装。

P7 secondLowestZeroBit: 利用公式 $\sim x \& (x + 1)$ 提取最低位的 0, 将其置为 1 后, 再次套用公式提取新的最低位 0。

P8 oddParity: 利用异或的“无进位加法”和结合律, 将 32 位分段(每 4 位)提取异或, 最后将结果逐步折叠 ($\gg 2$ 和 $\gg 1$) 至最低位, ! 取反得到奇偶校验位。

P9 rotateRightBits: 循环右移 n 位等价于 逻辑右移 n | 左移 (32-n)。利用 $(\sim n + 1) \& 31$ 替换 32-n 并规避 n=0 时的移位未定义行为。

P10 roundEvenPow2: 利用 $x + \text{bias} + \text{lsb}$ 的进位机制。 $\text{bias} = 2^{(n-1)} - 1$ 确保正中间默认舍去, $\text{lsb} = (x \gg n) \& 1$ 在商为奇数时补 1 进位, 实现向偶数舍入。

P11 midpointTowardFirst: 用 $\text{half_x} + \text{half_y} + (x \& y \& 1)$ 安全计算均值, 防止 x+y 溢出。通过分类讨论(同号/异号)安全计算 x-y 的符号位, 用 $(\text{mask} \& A) | (\sim \text{mask} \& B)$ 选出正确的修正值。

P12 isBetweenEitherOrder: 安全计算 x-a 和 x-b 的符号位。若符号位不同(异或得全 1), 则 x 在区间内。最后用 $!(x^a) | !(x^b)$ 补丁处理端点情况。

P14: 核心思路是采用“分步相加 + 分类讨论”:

首先安全计算 $x + y$, 并利用符号位异或逻辑判断是否溢出。若溢出, 需推导出当前结果与真实数学和的“溢出量 k”(正溢出为 $\text{INT_MAX} + 1 + k$, 负溢出为 $\text{INT_MIN} - 1 - k$)。

随后将 k 与 z 进行安全比较。例如, 在正溢出情况下, 如果 $z \geq -k$, 说明数学和依然大于 INT_MAX , 返回 1; 否则返回 -1。非溢出情况则继续与 z 进行安全加法并再次判断溢出。

P15: 补上隐藏位(0x800000)后乘以 3。根据乘法结果是否达到进位阈值(如 0x2000000), 决定阶码是否加 1。若产生进位, 尾数需除以 4 并进行向偶数舍入; 若无进位, 尾数除以 2 并进行向偶数舍入。随后需重新分离出规格化的尾数, 并处理尾数进位导致的二次阶码溢出(变为无穷大)。

P16: 通过真实指数 E 进行数轴分类讨论:

$E < -1$ (绝对值 < 0.5), 直接返回带符号的 0;

$E == -1$ (绝对值在 $[0.5, 1.0)$), 若尾数为 0 (刚好 0.5) 向偶数舍入返回 0, 否则进位到 1.0;

$E \geq 23$, 本身即为整数, 原样返回;

常规区间 $0 \leq E < 23$, 提取尾数并截断, 根据被砍掉的位判断是否进位: 大于一半进位, 刚好一半时看向偶数舍入规则(看保留位最后一位是否为奇数)。

特别处理: 在 $E == 0$ (绝对值在 $[1.0, 2.0)$) 区间, 整数部分隐藏的 1 是奇数, 但 frac 右移后会被清零, 导致丢失奇偶性判断。因此该区间被单独提取, 若 $\text{frac} \geq 0x400000$ (即 ≥ 1.5) 直接进位到 2.0, 否则保持 1.0。

P17 float_i2f: 提取符号, 安全取绝对值, while 循环找最高有效位算阶码 $e+127$ 。尾数处理时, 若 $e > 23$ 需右移砍位, 利用 $\text{out} > \text{half}$ 或 $== \text{half}$ 配合 $\text{frac} \& 1$ 实现向偶数舍入, 最后处理尾数进位溢出 ($\text{frac} == (1 \ll 23)$) 并拼装。

P18 bitCount: 。构造 0x11111111 掩码将每 4 位作为一个整体, 利用 $(x \gg 1) \& \text{mask1}$ 等操作把不同位的 1 累加到 4 位空间中。随后逐步将 4 位整体合并为 8 位、16 位、32 位整体, 得到总数。

P19 bitReverse: 依次进行相邻 1 位、2 位、4 位、8 位、16 位的交换。使用掩码(如 0x55555555、0x33333333)配合 $\gg$ 和 $\ll$ 实现并行翻转。

对实验的感受和对助教的建议：

个人认为这 19 题中 14、15、16 题最难，甚至对比前后的题目有一种难度一下子跃升的感觉。建议以后可以将难度最高的题目统一放在整个 lab 的最后。（甚至有点太难了，是我的问题吗……）

部分DataLab经验总结

1. 基础掩码操作

最高位掩码生成： $1 \ll 31$ 生成 0x80000000。

异或的实现（德摩根定律）：仅用 $\sim$ 和 $\&$ 实现 $x \wedge y$ ，公式为 $\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$ 。

符号掩码： $x \gg 31$ 可生成全 0（正数）或全 1（负数）的掩码。（即**提取符号位**）

（依据：① $a \wedge b = (a \& \sim b) \mid (\sim a \& b)$ ② $\sim(\sim X \& \sim Y) = X \mid Y$ ）

求相反数： $\sim x + 1$ 。（用于减法）

条件返回（无分支选择/MUX）： $(mask \& A) \mid (\sim mask \& B)$ 。利用全 0 或全 1 的掩码，将不需要的部分清零后拼接。（ **$\mid$ 可用于拼接， $\&$ 可用于提取所需部分或者将所需部分清零，再配合 $\gg$ 和 $\ll$ ，可以灵活选择需要的部分进行操作**）

2. 字节与半字节操作

逻辑右移（一律用 0 补齐）：先执行算术右移（用符号位补齐，和逻辑右移的矛盾体现在对负数的操作上） $x \gg n$ ，再与掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 进行 $\&$ 运算，擦除高位补入的 1。

半字节交换：构造掩码 0x0F0F0F0F（通过 $0x0F \mid (0x0F \ll 8)$ 逐级拼接），分别提取高低 4 位并反向移位后，用 $\mid$ 拼装。（**要学会通过 0、f、2、3、5、7、8 构造掩码序列实现不同的保留或清除操作**）

```
mask = 0x0F | (0x0F << 8);
mask = mask | (mask << 16);
a = (x & mask) << 4;
b = ((x & ~mask) >> 4) & mask;
return a | b;
```

3. 位级技巧与分治

提取最低位的 0： $\sim x \& (x + 1)$ 。将最低位 0 置 1 后再次调用此公式，可提取第二个 0。

$first_zero = \sim x \& (x + 1)$ ；（**只有最低位 0 才会在进行 $\sim x$ 和 $(x + 1)$ 后都会变成 1**，两个操作 $\&$ ，卡出结果为除了最低位 0 的位置变为 1 以外、其他位置变为 0 的一个数）

$x2 = x \mid first_zero$ ；（拼接，将最低位 0 变为 1，其余不变）

$second_zero = \sim x2 \& (x2 + 1)$ ；

奇偶校验：利用异或的无进位加法与结合律。将 32 位分块异或，逐步折叠（ $\gg 16, 8, 4, 2, 1$ ），最终最低位代表 1 的个数奇偶性。**返回值 1 表示 x 的二进制中 1 的个数是偶数；返回 0 表示 1 的个数是奇数。**

```

int a = (x >> 28) & 0xF;
int b = (x >> 24) & 0xF;
int c = (x >> 20) & 0xF;
int d = (x >> 16) & 0xF;
int e = (x >> 12) & 0xF;
int f = (x >> 8) & 0xF;
int g = (x >> 4) & 0xF;
int h = x & 0xF;
int y = a ^ b ^ c ^ d ^ e ^ f ^ g ^ h;

```

(y的每一位分别表示)

第 0 位: 所有 nibble 第 0 位的奇偶

第 1 位: 所有 nibble 第 1 位的奇偶

第 2 位: 所有 nibble 第 2 位的奇偶

第 3 位: 所有 nibble 第 3 位的奇偶

```
y = y ^ (y >> 2);
```

```
y = y ^ (y >> 1);
```

这两步是在把 y 的 4 个位不断折叠异或, 最终让最低位变成 y 四个位的异或结果。

```
return (y & 1);
```

循环右移 (把一个固定位数的二进制数整体向右移动, 移出去的低位不丢弃, 而是补到最高位):

```
int a = (x >> n) & ~(((1 << 31) >> n) << 1); (右移n位并清除符号扩展)
```

```
int b = x << ((~n + 1) & 31); (注意: -n用~n + 1代替) ((~n + 1) & 31等价于 (32 - n) % 32, 但比写32-n安全, 因为当n=0时, 左移32位是非法的)
```

(一个数 x 对 2^k 取余, 结果就是 x 的二进制表示中最低 k 位所代表的值。31的二进制表示是11111, 与31&, 就是截取最低五位。)

```
return a|b;
```

将整数 x 舍入到最近的 2^n 的倍数:

采用银行家舍入 (round half to even) 规则, 即:

如果 $r < 2^{(n-1)}$: 舍去, 结果 $q * 2^n$ 。

如果 $r > 2^{(n-1)}$: 进位, 结果 $(q+1) * 2^n$ 。

如果 $r == 2^{(n-1)}$: 恰好一半, 舍入到偶数倍的 2^n 。

```
int quo = (x>>n)&1;
```

(取出 x 右移 n 位后的最低位, 也就是 x 的第 n 位 (从 0 开始计数)。如果令 $x = q * 2^n + r$ (其中 $0 \leq r < 2^n$), 那么 $x \gg n = q$, 所以 **quo = q & 1, 即商 q 的奇偶性**)

```
int bias = (1<<(n+~0))+~0;
```

(~0 是 -1。n + ~0 就是 n - 1。1 << (n - 1) 等于 $2^{(n-1)}$ 。再加 ~0 (即 -1), 得到 $bias = 2^{(n-1)} - 1$ 。这是标准四舍五入到 2^n 倍数时常用的偏置值 (比一半少 1))

```
return ((quo + bias + x)>> n) << n;
```

(右移再左移相当于把低 n 位清零, 也就是向下取整到 2^n 的倍数, 若原来 $A = q * 2^n + r$, 则操作后低n位清零, $A' = q * 2^n$)

·为什么这样可以实现银行家舍入?

情况一: q 是偶数 (如果要舍入, 则向下舍入到 $q * 2^n$)

```
quo = q & 1 = 0, bias = 2^(n-1) - 1
```

进位条件: $r + (2^{(n-1)} - 1) + 0 \geq 2^n \Rightarrow r + 2^{(n-1)} - 1 \geq 2^n \Rightarrow r \geq 2^n - 2^{(n-1)} + 1 \Rightarrow r \geq 2^{(n-1)} + 1$, 因为r是整数, 所以相当于 $r > 2^{(n-1)}$ 。当 $r > 2^{(n-1)}$ 时, 进位; 当 $r = 2^{(n-1)}$ 时, 不进位, 结果保持 $q * 2^n$ 。

情况二: q 是奇数 (如果要舍入, 则向上舍入到更大的 2^n 的偶数倍)

```
quo = 1, bias + quo = (2^(n-1) - 1) + 1 = 2^(n-1)
```

进位条件: $r + 2^{(n-1)} \geq 2^n \Rightarrow r \geq 2^{(n-1)}$ 。当 $r = 2^{(n-1)}$ 时, 进位, 结果变成 $(q+1) * 2^n$ 。因为 q 是奇数, $q+1$ 就是偶数, 所以 $(q+1) * 2^n$ 是偶数倍的 2^n 。

所以, 是否进位完全取决于 $r + bias + quo$ 是否达到 2^n 。

统计二进制中 1 的个数: 核心思想: 分治法 / 分桶累加。将一个 32 位整数, 逐级切分成小块统计, 最后合并。

```
mask1 = (0x11<<8)|0x11;
```

```
mask1 = mask1|(mask1<<16);
```

(mask1=0001 0001 0001 0001 0001 0001 0001 0001, 四次提取的结果相加, 就把每个 nibble 中的 4 个比特全部累加。因为每个比特是 0 或 1, 所以每个 nibble 的和在 0~4 之间, 不会进位到相邻 nibble。)

```
sum1 = (x&mask1)+((x>>1)&mask1)+((x>>2)&mask1)+((x>>3)&mask1);
```

```
mask2 = (0x0f<<8)|0x0f;
```

```
mask2 = (mask2<<16)|mask2;
```

```
sum2 = (sum1&mask2)+((sum1>>4)&mask2);
```

(把每个字节中两个 nibble 的计数合并, 结果存放在每个字节的低 4 位。因为每个 nibble 计数最大为 4, 两个相加最大为 8, 需要 4 位表示, 所以不会溢出到高 4 位。)

```
mask3 = 0xFF | (0xFF << 16);
```

```
sum3 = (sum2 & mask3) + ((sum2 >> 8) & mask3);
```

(两者相加, 把每个 16 位中两个字节的计数合并, 结果存放在每个 16 位的低 8 位。每个字节计数最大为 8, 两个相加最大为 16, 需要 5 位, 但低 8 位足够容纳。)

```
mask4 = 0xFF | (0xFF << 8);
```

```
sum4 = (sum3 & mask4) + ((sum3 >> 16) & mask4);
```

```
return sum4;
```

二进制位翻转:

```
re1 = ((x>>1)&0x55555555)|((x&0x55555555)<<1); (交换相邻的一位)
```

```
re2 = ((re1>>2)&0x33333333)|((re1&0x33333333)<<2); (交换相邻的二位组)
```

```
re3 = ((re2>>4)&0x0f0f0f0f)|((re2&0x0f0f0f0f)<<4); (交换相邻的半字节)
```

```
re4 = ((re3>>8)&0x00ff00ff)|((re3&0x00ff00ff)<<8); (交换相邻的字节)
```

```
re5 = ((re4>>16)&0x0000ffff)|((re4&0x0000ffff)<<16); (交换相邻的半字)
```

(经过这五步, 整个 32 位整数的位顺序被完全反转)

```
return re5;
```

4. 安全数学运算与溢出

安全**求平均值** (不能直接 $(x+y)>>1$, 可能会发生溢出):

$(x \& y) + ((x \wedge y) >> 1)$

($x \wedge y$: 找出所有两个数不同的位, 在这些位上做加法后不产生进位。 $x \& y$: 找出所有两个数同时为 1 的位, 这些位相加时会产生进位。 $<< 1$: 因为进位是加到高一位的, 且原来的位上归 0, 所以要把这些进位整体左移一位。因此, $x + y = (x \wedge y) + ((x \& y) << 1)$ 。再除以二就是上式。)

或 $(x >> 1) + (y >> 1) + (x \& y \& 1)$

($x >> 1$ 和 $y >> 1$ 分别把 x 和 y 除以 2。但如果 x 和 y 都是奇数, 它们各自除以 2 都会丢掉一个 0.5, 总和就丢了 1。 $x \& y \& 1$ 就是用来检测“两者是否都是奇数”, 如果是, 结果就是 1, 补上这个丢失的 1。)

安全**判断大小**: 不可直接使用减法。若符号不同, 正数必大于负数 (通过异或符号位获取掩码) $x-y$ 结果的符号与 x 相同; 若符号相同, $x-y$ 不会溢出, 直接取差值符号位。判断区间: 乘 5 饱和截断:

计算 $x * 5$ 拆解为 $(x << 2) + x$ 。

```
int x4 = x << 2;
```

```
int x5 = x4 + x;
```

```
int ov_shift = (!((x >> 29) ^ (x >> 31))) << 31 >> 31; (!:把任意非零值变成 1, 把 0 变成 0。)
```

(**检测左移溢出**: $x << 2$ 把 x 左移 2 位, 结果的符号位来自 x 的第 29 位。如果第 29 位与原来的符号位 (第 31 位) 不同, 则左移后符号位改变, 表示溢出。 $x >> 31$: 算术右移 31 位, 得到符号位扩展 (全 0 或全 1, 也就是全部为原数的第 31 位。)。 $x >> 29$: 算术右移 29 位, 最低 3 位分别是原 x 的第 31、30、29 位, 其余位数为原数的符号位填充。 $(x >> 29) ^ (x >> 31)$: 比较第 29 位与第 31 位是否不同 (最低位为 位 29 ^ 位 31), 以及第 30

位与第 31 位是否不同（第二位为 位30 ^ 位31）。!! 将结果转为 0 或 1：只要有一位不同，就为 1（表示溢出）。<< 31 >> 31：将 1 变成全 1（-1），将 0 变成全 0，作为溢出掩码。）

```
int ov_add = ~(x4 ^ x) & (x4 ^ x5) >> 31;
```

（有符号加法溢出检测：对于 $a + b = c$ ，如果 a 和 b 符号相同，而 c 与它们符号不同，则溢出。 $\sim(x4 \wedge x)$ 最高位为 1 表示 $x4$ 和 x 符号相同， $(x4 \wedge x5)$ 最高位为 1 表示 $x5$ 与 $x4$ 符号不同。两者按位与后，最高位为 1 当且仅当： $x4$ 和 x 符号相同，并且 $x5$ 与 $x4$ 符号不同。 $\gg 31$ ：算术右移 31 位，把最高位扩展到所有位，得到全 1（溢出）或全 0（未溢出）。所以 `ov_add` 是加法溢出的标志掩码。）

```
int overflow = ov_shift | ov_add;
```

（综合溢出标志：只要左移或加法任意一步溢出，`overflow` 就是全 1。）

```
int INT_MIN = 1 << 31;
```

```
int INT_MAX = ~INT_MIN;
```

```
int x_sign = x >> 31;
```

```
int INT = (x_sign & INT_MIN) | (~x_sign & INT_MAX);
```

（`INT_MIN` 是 32 位有符号整数的最小值。`INT_MAX` 是最大值。`x_sign` 是 x 的符号掩码。`INT` 根据 x 的符号选择饱和值。如果 $x < 0$ ，`x_sign` 全 1， $\sim x_sign$ 全 0，`INT = INT_MIN`。如果 $x \geq 0$ ，`x_sign` 全 0， $\sim x_sign$ 全 1，`INT = INT_MAX`。）

```
return (overflow & INT) | (~overflow & x5);
```

（根据是否溢出选择饱和值或者返回运算结果）

饱和值选择：正数溢出返回 `INT_MAX (0x7fffffff)`，负数溢出返回 `INT_MIN (0x80000000)`。

5.浮点数结构

单精度浮点数 32 位由三部分组成：

符号位（1位）：最高位。

阶码（8位）： E （真实指数，表示 2^E ）+ 127。全 0 表示非规格化数或 0，全 1 表示无穷大或 NaN。

尾数（23位）：存科学计数法小数点后的部分。从左到右的位次单位，依次表示 $2^{-1}, 2^{-2}, 2^{-3}, \dots$ 。

真实数值 = 1.尾数 * 2^E （规格化数）。

6.整数转浮点数

提取符号，取绝对值。

通过 while 循环找到最高有效位 e （从左到右找第一个出现的 1），阶码 $\text{exp} = e + 127$ 。

```
if (x == 0) return 0;
```

```
sign = x & 0x80000000;
```

```
ux = x;
```

```
if (x < 0) ux = -ux;
```

```
e = 31;
```

```
while (!(ux & (1u << e))) {
```

```
    e--;
```

```
}
```

尾数处理：若 $e \leq 23$ 则左移对齐；若 $e > 23$ 则右移砍掉多余位。（并时刻与 `0x7fffff`，也就是 111...111（23 个 1），进行 & 运算清除前面因为右移导致的补齐）

向偶数舍入：被砍掉的位 `round_bits` 与一半 `half` 比较。大于一半进位，等于一半时看保留的最低位（`frac & 1`），为奇数则进位。

处理尾数进位溢出（尾数变成 $1 << 23$ 时，说明向 `exp` 范围进了 1 且同时将 `frac` 范围清零，阶码加 1，尾数清零）。

```
if (e <= 23) {
```

```
    frac = (ux << (23 - e)) & 0x7fffff;
```

```
} else {
```

```
    frac = (ux >> (e - 23)) & 0x7fffff;
```

```
    mask = (1 << (e - 23)) - 1;
```

```

out = mask & ux;
half = 1 << (e - 24); (也即1+ (e-23-1) 个0, 也即 (e-23) 个1的一半)
if (out > half) {
    frac++;
} else if (out == half) {
    if (frac & 1) frac++;
}
}
if (frac == (1 << 23)) {
    frac = 0;
    exp++;
}

```

7.浮点数舍入到整数

舍入规则采用“向偶数舍入”：小数部分 < 0.5：舍去；小数部分 > 0.5：进位；小数部分 = 0.5：舍入到最近的偶数。

分类讨论依据（真实指数 E ）：

$E < -1$ （绝对值严格 < 0.5，应直接舍入到0）：直接返回 sign（也即带符号的 0）。

$E == -1$ （绝对值在 [0.5, 1.0)）：若尾数为 0（即该数值为0.5）：，向偶数舍入返回 0；否则（该数值>0.5）返回 1.0 的表示（sign | (127 << 23)）。

$E \geq 23$ ：本身就是整数，直接返回原值。

$E == 0$ （绝对值在 [1.0, 2.0)）：此时数值 = $2^0 * (1 + f/2^{23}) = 1 + f/2^{23}$ ，范围 [1.0, 2.0)。

注意：整数部分隐藏的 1 是奇数。如果直接用 $f \gg 23$ 来提取整数部分，会得到 0，这样会丢失隐藏的整数 1，所以需要特殊处理。若尾数 $\geq 0x400000$ （即该数值 ≥ 1.5 ），进位为 2.0（128 << 23），否则为 1.0（127 << 23）。

$0 < E < 23$ （常规舍入）：shift = 23 - E。整数部分 = $2^E + (frac \gg shift)$ ，其中 shift = 23 - E。小数部分 = 被移出的低 shift 位。提取 round_bits（即 f 的低 shift 位）和 half（即 1 << (shift - 1)，代表 0.5 的阈值）。将 f 右移 shift 位，得到 $f_shifted = f \gg shift$ 。如果移出的部分 > half，则进位。如果移出的部分 < half，则舍去。如果移出的部分 == half，则向偶数舍入：看当前整数部分的最低位（即 $f_shifted$ 的最低位，或者整体整数部分的最低位）是否为奇数，若为奇数则进位，否则舍去。

处理溢出：如果进位导致 $f_shifted$ 达到 $1 \ll E$ ，即整数部分从 2^E 变成了 $2^{(E+1)}$ ，需要调整指数。描述中“处理溢出（ $frac == 1 \ll 23$ ）”可能是指在某些实现中，进位后尾数可能溢出到指数位，需要特殊处理。